

TD5 : Mise en œuvre d'une Pipeline CI/CD avec GitHub Actions et AWS

Partie 1 : Intégration Continue

On commence par installer tous les prérequis nécessaires

```
PS C:\WINDOWS\system32> kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready    control-plane   59m   v1.34.0
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
PS C:\WINDOWS\system32> kubectl config current-context
minikube
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> flux --version
flux version 2.7.5
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> tofu version
OpenTofu v1.11.3
on windows_amd64
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> node --version
v24.13.0
PS C:\WINDOWS\system32> npm --version
11.6.2
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> aws --version
aws-cli/2.33.2 Python/3.13.11 Windows/11 exe/AMD64
PS C:\WINDOWS\system32> aws sts get-caller-identity
{
  "UserId": "AIDA6POG2RPQUDTV5XPX5",
  "Account": "995238775777",
  "Arn": "arn:aws:iam::995238775777:user/devopsYacine"
```

1.1 Préparation du Projet et Tests Automatisés

La première étape consiste à préparer une application Node.js et à configurer des tests unitaires automatisés.

On prépare le projet :

Nom	Modifié le	Type	Taille
aws	26/01/2026 23:38	Dossier de fichiers	
coverage	26/01/2026 23:39	Dossier de fichiers	
node_modules	26/01/2026 23:40	Dossier de fichiers	
.gitignore	26/01/2026 23:38	Fichier source Git L	1 Ko
app.js	26/01/2026 23:38	JSFile	1 Ko
app.test.js	26/01/2026 23:38	JSFile	3 Ko
awscli2	26/01/2026 23:38	Dossier compressé	65 309 Ko
build-docker-image	26/01/2026 23:38	Fichier source SH	1 Ko
Dockerfile	26/01/2026 23:38	Fichier	1 Ko
package	26/01/2026 23:38	Fichier source JSON	1 Ko
package-lock	26/01/2026 23:40	Fichier source JSON	203 Ko
run-docker	26/01/2026 23:38	Fichier source SH	1 Ko
server.js	26/01/2026 23:38	JSFile	1 Ko

Cela fait, je crée le Github Actions workflow que je push sur github :

```

Administrateur : Windows PowerShell
S C:\Users\yacine.oualiken\Desktop\devops-td5> mkdir -p .github/workflows

Répertoire : C:\Users\yacine.oualiken\Desktop\devops-td5\.github

ode                LastWriteTime         Length Name
---                -
-----          26/01/2026    23:53          workflows

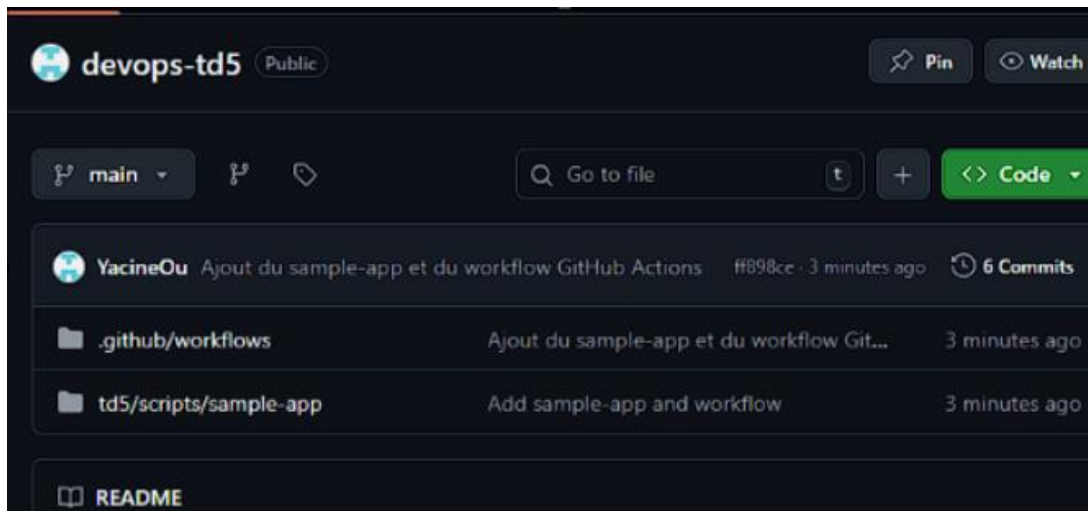
S C:\Users\yacine.oualiken\Desktop\devops-td5> cd .github/workflows
S C:\Users\yacine.oualiken\Desktop\devops-td5\.github\workflows> notepad app-tests.yml
S C:\Users\yacine.oualiken\Desktop\devops-td5\.github\workflows>

app-tests.yml
Fichier  Modifier  Affichage

# .github/workflows/app-tests.yml (Example 5-2)
name: Sample App Tests
on: push
jobs:
  sample_app_tests:
    name: "Run Tests Using Jest"
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3 - name: Install dependencies
        working-directory: td5/scripts/sample-app
      - run: npm install - name: Run tests
        working-directory: td5/scripts/sample-app
      - run: npm test

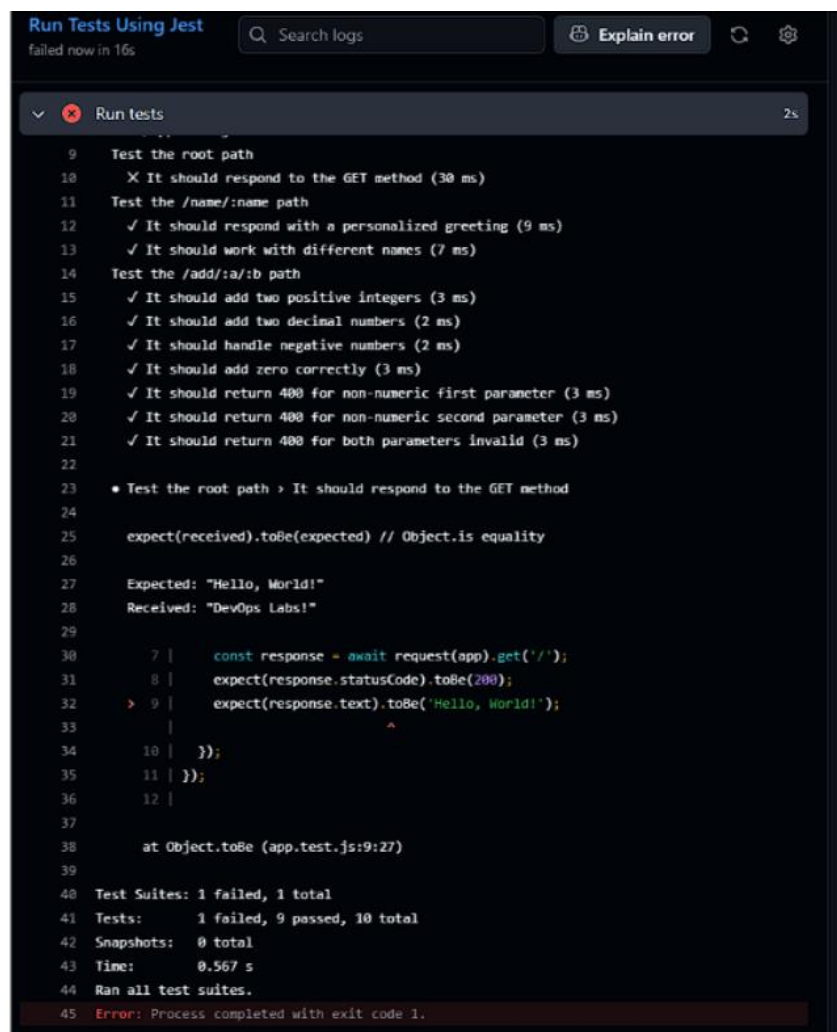
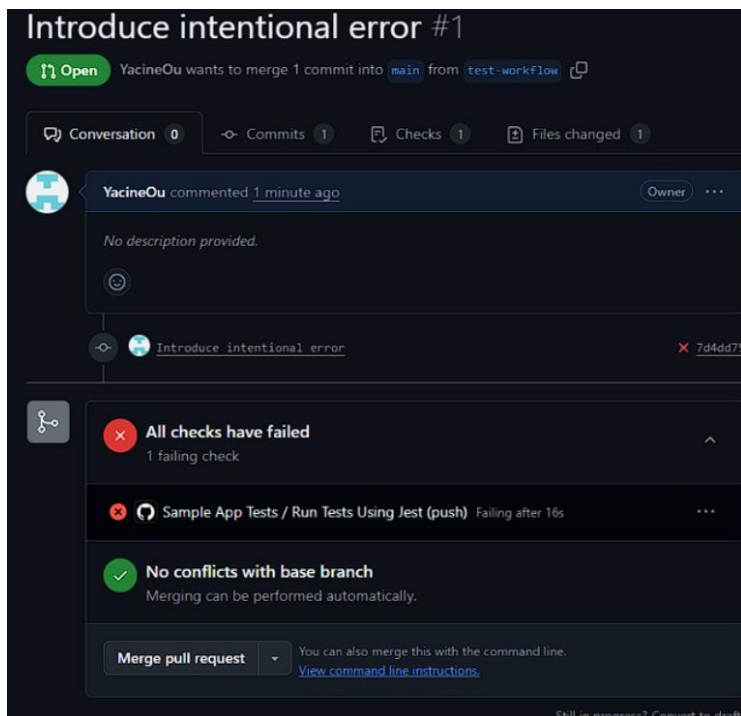
Ln 4, Col 7  363 caractères  Texte brut  100%  Windows (CRLF)  UT

```



J'y introduit une erreur intentionnellement :

```
PS C:\Users\yacine.oualiken\Desktop\devops-td5> git checkout -b test-workflow
Switched to a new branch 'test-workflow'
PS C:\Users\yacine.oualiken\Desktop\devops-td5> git add td5/scripts/sample-app/app.js
PS C:\Users\yacine.oualiken\Desktop\devops-td5> git commit -m "Introduce intentional error"
[test-workflow 7d4dd75] Introduce intentional error
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\yacine.oualiken\Desktop\devops-td5> git push origin test-workflow
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 484 bytes | 484.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'test-workflow' on GitHub by visiting:
remote:   https://github.com/YacineOu/devops-td5/pull/new/test-workflow
remote:
To https://github.com/YacineOu/devops-td5.git
 * [new branch]      test-workflow -> test-workflow
PS C:\Users\yacine.oualiken\Desktop\devops-td5>
```

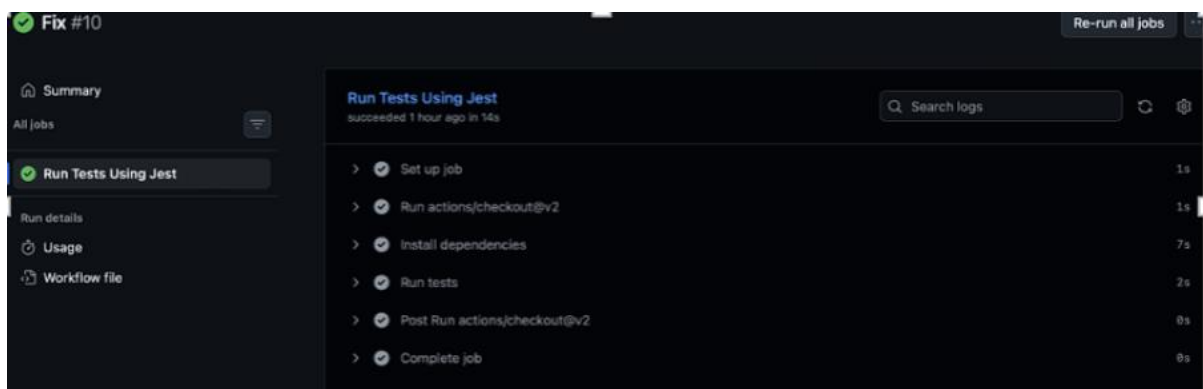
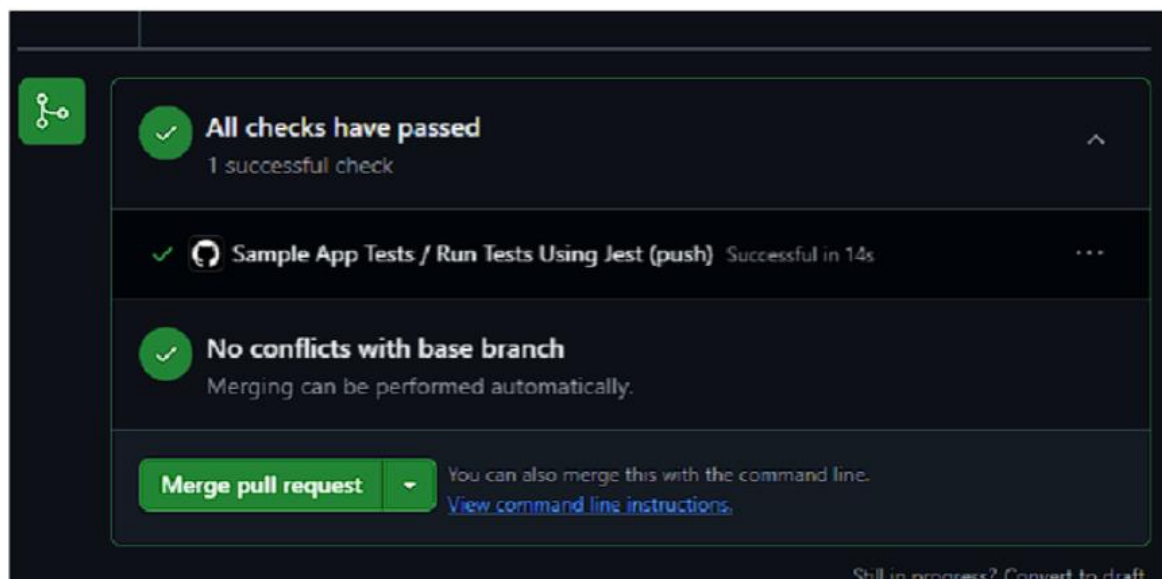


Erreur que je corrige juste après :

```

PS C:\Users\yacine.oualiken\Desktop\devops-td5> git add td5/scripts/sample-app/app.test.js
PS C:\Users\yacine.oualiken\Desktop\devops-td5> git commit -m "Update response text in test"
[test-workflow d13fd2b] Update response text in test
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\yacine.oualiken\Desktop\devops-td5> git push origin test-workflow
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 487 bytes | 487.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/YacineOu/devops-td5.git
7d4dd75..d13fd2b test-workflow -> test-workflow
PS C:\Users\yacine.oualiken\Desktop\devops-td5>

```



Workflow s'est exécuté avec succès, le code ne contient pas de régression avant d'être fusionné.

1.2 Configuration d'OIDC avec AWS

Pour permettre à GitHub Actions d'interagir avec AWS sans stocker de clés d'accès permanentes, nous avons configuré OpenID Connect .

On crée un main.tf pour récupérer les autorisations sur aws :

```
main.tf  outputs.tf  +
Fichier  Modifier  Affichage

# Fournisseur AWS
provider "aws" {
  region = "us-east-2"
}

# Module OIDC pour GitHub Actions
module "oidc_provider" {
  source      = "../../modules/github-aws-oidc"
  provider_url = "https://token.actions.githubusercontent.com"
}

# Module IAM Roles pour GitHub Actions
module "iam_roles" {
  source = "../../modules/gh-actions-iam-roles"

  name                = "lambda-sample"
  oidc_provider_arn    = module.oidc_provider.oidc_provider_arn

  enable_iam_role_for_testing = true
  enable_iam_role_for_plan    = true
  enable_iam_role_for_apply   = true

  github_repo      = "YacineOu/devops-td5"
  lambda_base_name = "lambda-sample"
  tofu_state_bucket = "devops-td5-state-bucket"
  tofu_state_dynamodb_table = "devops-td5-locks"
}
```

devopsYacine infos

Supprimer

Récapitulatif

ARN
[arn:aws:iam::995238775777:user/devopsYacine](#)

Création
January 18, 2026, 18:47 (UTC+01:00)

Accès par console
[Activé sans l'authentification MFA](#)

Dernière connexion à la console
[Aujourd'hui](#)

Clé d'accès 1
[AKIA6POG2RPQY7GGK7XB - Active](#)
[Utilisé il y a 24 jours. 32 jours anciens.](#)

Clé d'accès 2
[Créer une clé d'accès](#)

[Autorisations](#) | [Groupes](#) | [Balises](#) | [Informations d'identification de sécurité](#) | [Dernier accès](#)

Politiques des autorisations (1)

Les autorisations sont définies par des politiques attachées à l'utilisateur directement ou via des groupes.



Supprimer

[Ajouter des autorisations](#)

Rechercher

Filtrer par Type

Tous les types

< 1 > ⚙

☐ Nom de la politique ⓘ

▲ | Type

▼ | Attaché via ⓘ

☐ [AdministratorAccess](#)

Gérées par AWS – fonction professionnelle

Directement

► Limite d'autorisations (non défini)

▼ Générer une politique basée sur les événements CloudTrail

Vous pouvez générer une nouvelle politique basée sur l'activité d'accès pour utilisateur, puis la personnaliser, la créer et l'attacher à ce rôle. AWS utilise vos événements CloudTrail pour identifier les services et actions utilisés et générer une politique. [En savoir plus ⓘ](#)

On initialise tofu et pour créer nos clé secrètes

```

PS C:\Users\yacine.oualiken\Desktop\devops-td5\td5\scripts\tofu\live\ci-cd-permissions> tofu init

Initializing the backend...
Initializing modules...
- iam_roles in ../../modules\gh-actions-iam-roles
- oidc_provider in ../../modules\github-aws-oidc

Initializing provider plugins...
- Reusing previous version of hashicorp/tls from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Installing hashicorp/tls v4.0.5...
- Installed hashicorp/tls v4.0.5 (signed, key ID 0C0AF313E5FD9F80)
- Installing hashicorp/aws v5.51.1...
- Installed hashicorp/aws v5.51.1 (signed, key ID 0C0AF313E5FD9F80)

Providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://opentofu.org/docs/cli/plugins/signing/

OpenTofu has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

OpenTofu has been successfully initialized!

You may now begin working with OpenTofu. Try running "tofu plan" to see
any changes that are required for your infrastructure. All OpenTofu commands
should now work.

If you ever set or change modules or backend configuration for OpenTofu,
rerun this command to reinitialize your working directory. If you forget, other

```

```

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

lambda_deploy_apply_role_arn = "arn:aws:iam::995238775777:role/lambda-sample-apply"
lambda_deploy_plan_role_arn = "arn:aws:iam::995238775777:role/lambda-sample-plan"
lambda_test_role_arn = "arn:aws:iam::995238775777:role/lambda-sample-tests"

```

```

lambda_deploy_apply_role_arn = "arn:aws:iam::995238775777:role/lambda-sample-apply"
lambda_deploy_plan_role_arn = "arn:aws:iam::995238775777:role/lambda-sample-plan"
lambda_test_role_arn = "arn:aws:iam::995238775777:role/lambda-sample-tests"

```

1.3 Tests d'Infrastructure

Une fois les permissions configurées, nous avons mis en place des tests pour vérifier que notre infrastructure est correctement définie avant tout déploiement.

On copie nos clés sur aws :

Actions secrets / Update secret

TEST_ROLE_ARN

Value

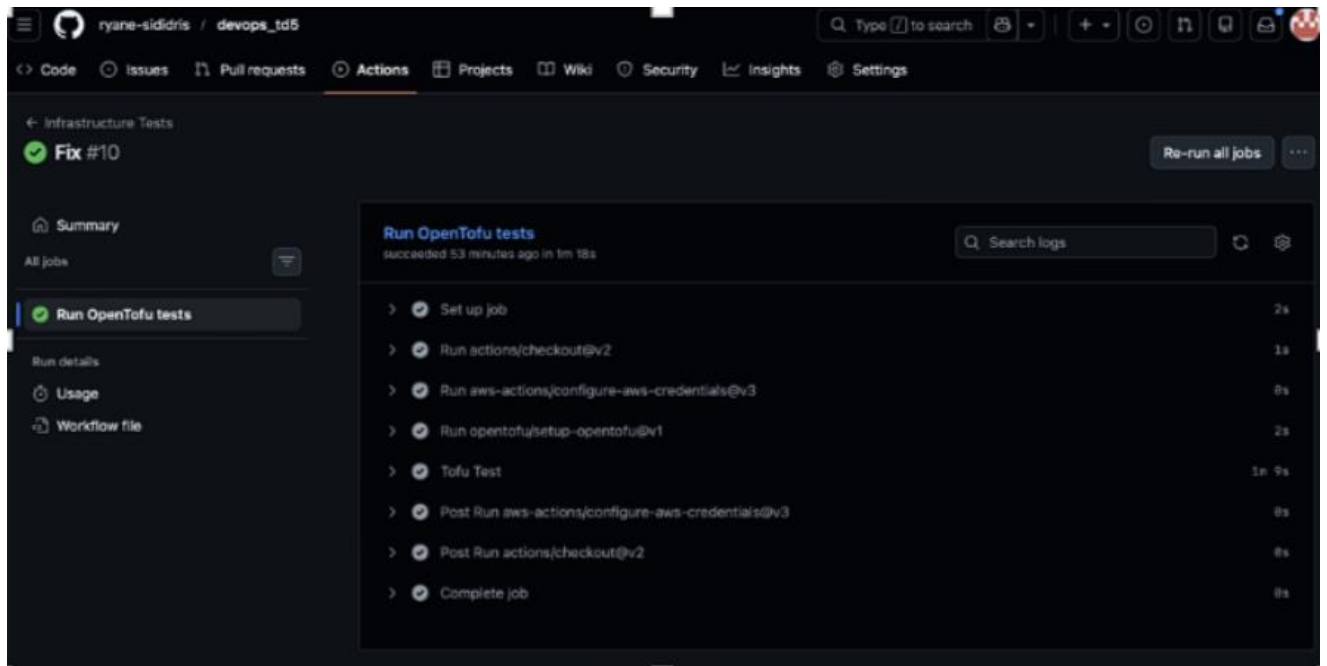
```
arn:aws:iam::995238775777:role/lambda-sample-tests
```

Update secret

Puis mettons en place le Role-To-Assume changer :

```
infra-tests.yml
Fichier  Modifier  Affichage

# name: Infrastructure Tests
on: push
jobs:
  opentofu_test:
    name: "Run OpenTofu tests"
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v2
      - uses: aws-actions/configure-aws-credentials@v3
    with:
      # TODO: fill in your IAM role ARN!
      role-to-assume: ${ secrets.TEST_ROLE_ARN }
      role-session-name: tests-${ github.run_number }-${ github.actor }
      aws-region: us-east-2
      - uses: opentofu/setup-opentofu@v1
      - name: Tofu Test
    env:
      TF_VAR_name: lambda-sample-${ github.run_id }
      working-directory: td5/scripts/tofu/live/lambda-sample
    run: |
      tofu init -backend=false -input=false
      tofu test -verbose
```



GitHub Actions valide les fichiers de configuration OpenTofu, garantissant une création des ressources cloud sans erreur de syntaxe

Partie 2 : Livraison Continue

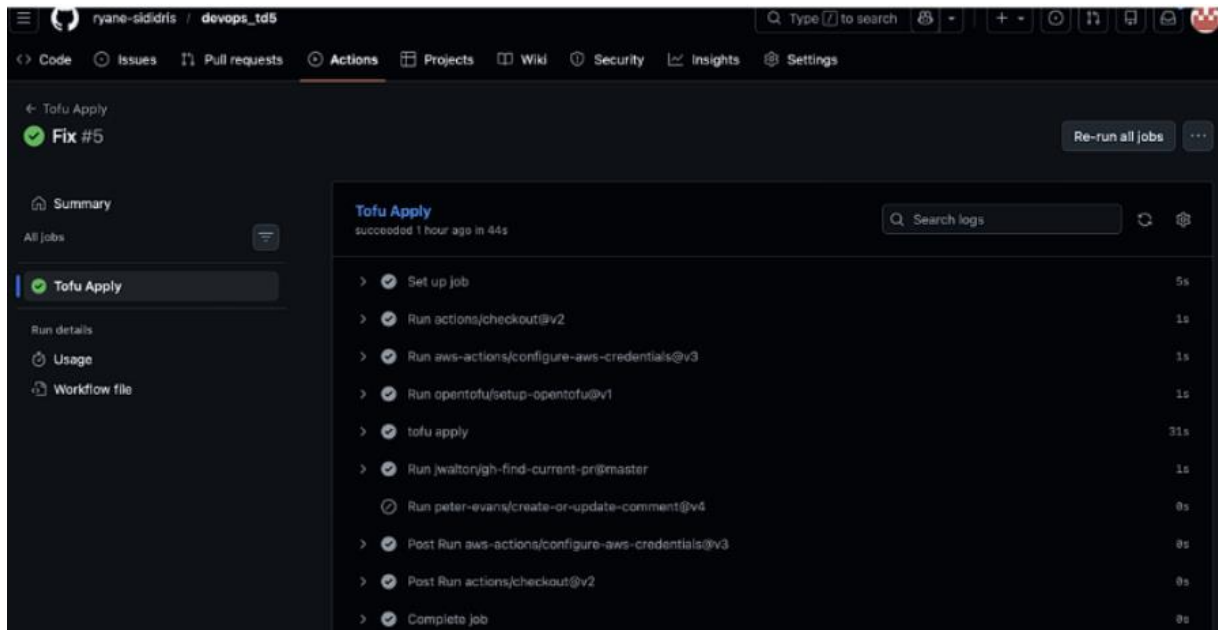
2.1 Backend distant pour l'état OpenTofu

On configure un backend distant pour stocker le fichier d'état de notre infrastructure. Cela évite les conflits lorsque plusieurs personnes modifient l'infrastructure.

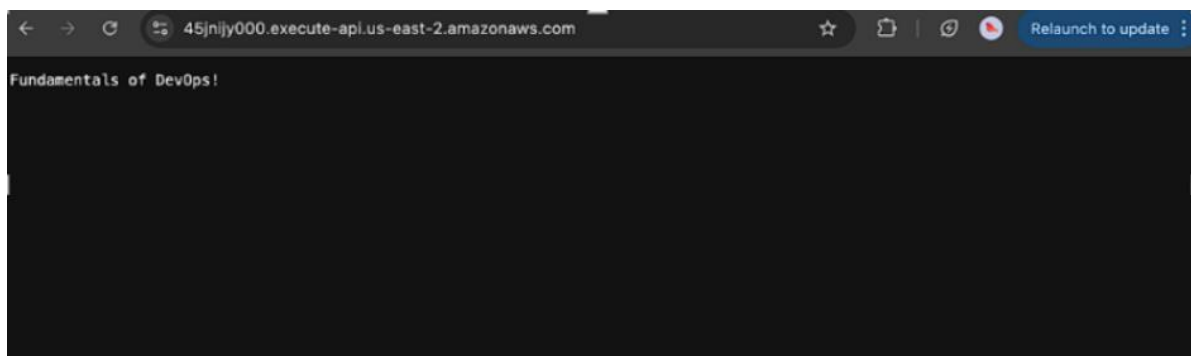
```
Users > rsd > Downloads > td5 > scripts > tofu > live > lambda-sample > backend.tf
1  terraform {
2      backend "s3" {
3          bucket      = "ryan-bucket-state-td5-fin"
4          key          = "devops-td5/lambda-sample/terraform.tfstate"
5          region       = "us-east-2"
6          encrypt      = true
7      }
8  }
```

2.2 Workflow de déploiement

Nous avons mis en place un pipeline de déploiement automatisé qui débute par la génération d'un plan via la commande "Tofu plan" lequel est ensuite publié sous forme de commentaire dans la Pull Request. Enfin, le workflow déclenche l'application des changements avec la commande "tofu apply"



En accédant à l'URL fournie par AWS, on peut voir s'afficher



Le succès de l'automatisation est confirmé par le déploiement de la fonction Lambda via OpenTofu. L'URL générée est active et répond aux requêtes HTTP prouvant que le workflow a mis à jour le code source avec succès.

Conclusion et Note Personnelle

Ce TD m'a fait me rendre compte à quel point l'automatisation est devenue indispensable aujourd'hui. On gagne énormément en temps et en fiabilité quand on passe de tests faits à la main à une pipeline complète qui gère tout toute seule : la sécurité avec OIDC, la validation via la CI et le déploiement final

Note Personnelle (Yacine OUALIKEN) : J'ai rencontré d'importantes difficultés lors de la partie 1.5 concernant la configuration OIDC. J'ai eu de nombreux problèmes de permissions sur AWS (probablement dû à une mauvaise configuration initiale de mes politiques IAM / relation de confiance). Après plusieurs tentatives infructueuses seul, la deuxième partie du TD a été réalisée avec Ryane. Nous avons repris le travail de A à Z sur sa machine pour finaliser les configurations et réussir le déploiement.